

# ELTS Repository, Portability, and Multi-LLM System Architecture Specification

*Normative design and coding-agent bootstrap manual*

| Field                 | Value                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Document ID           | SPEC-ELTS-ARCH-001                                                                                                            |
| Version               | 1.0.0                                                                                                                         |
| Status                | Implementation baseline                                                                                                       |
| Baseline date         | 4 September 2026                                                                                                              |
| Repository            | <a href="https://github.com/AceeEcho/DESS-ELTS-Research-Software">https://github.com/AceeEcho/DESS-ELTS-Research-Software</a> |
| Companion deliverable | ELTS Implementation and Progress Workbook                                                                                     |
| Source plan           | ELTS Build & Integration Plan v1.0                                                                                            |

**Purpose:** Attach this specification and its companion workbook to the first Claude Code, Codex, or comparable coding-agent prompt. Together they define what to build, how the repository must be governed, and how the exact current step is recorded.

**Repository:** [GitHub repository](#)

## How to Use This Specification

This document is the architecture and governance authority. The companion workbook is the execution catalog and operational tracking surface. An implementing agent must preserve both source files unchanged under project-management/baseline/, normalize the workbook into machine-readable data, create the progress reducer, and then continue from the exact step reported in PROJECT\_STATE.json.

| Reader               | Required action                                                                                                                       |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Initial coding agent | Read sections 1-4, 6-9, 14-17, and 21 before editing. Execute section 21 exactly within the authority hierarchy.                      |
| Developer            | Use the task and atomic-step IDs in the workbook. Work on one temporary branch, produce evidence, and update progress through events. |
| Investigator         | Own research, safety, and study-design decisions. Approve hard gates and frozen study releases.                                       |
| Reviewer             | Check scientific invariants, affected module contracts, tests, evidence, documentation, and generated-state consistency.              |
| Operator             | Use released operator procedures and diagnostics. Do not run participant sessions from the Unity Editor or a dirty build.             |

**Initial exact step:** P0.3.S001 - inventory the repository, record the baseline, and preserve existing work. The agent must not skip this because the remote repository may have changed since this document was authored.

## Authority and precedence

1. Approved research protocol, IRB constraints, and physical safety controls.
2. This architecture specification.
3. Accepted architecture decision records (ADRs), including decisions D-01 through D-18 without renumbering.
4. Versioned module contracts, schemas, and frozen study configuration.
5. The companion workbook and its normalized task catalog.
6. Implementation notes and code comments.

If two authorities conflict, work stops. The conflict is recorded as a blocker event and is resolved by the appropriate human owner; an agent may not silently choose the lower authority.

## Contents

| Section | Subject                                            |
|---------|----------------------------------------------------|
| 1       | Document control and normative language            |
| 2       | Purpose, governing principle, and baseline         |
| 3       | Scope and non-goals                                |
| 4       | Scientific and safety invariants                   |
| 5       | System context and component boundaries            |
| 6       | Canonical monorepo architecture                    |
| 7       | Portability and provisioning                       |
| 8       | Toolchain, dependency locking, and reproducibility |

| Section        | Subject                                                                                |
|----------------|----------------------------------------------------------------------------------------|
| 9              | Configuration and schema architecture                                                  |
| 10             | Unity application architecture                                                         |
| 11             | ELTS Jetson integration                                                                |
| 12             | Logging, research data, and analysis                                                   |
| 13             | Build, CI, diagnostics, and release                                                    |
| 14             | GitHub and multi-machine workflow                                                      |
| 15             | Documentation and ADR governance                                                       |
| 16             | Multi-LLM and multi-agent protocol                                                     |
| 17             | Machine-readable plan and progress state                                               |
| 18             | Implementation sequence and hard gates                                                 |
| 19             | Security, privacy, and safety controls                                                 |
| 20             | Verification and acceptance                                                            |
| 21             | Initial coding-agent directive                                                         |
| Appendices A-F | Repository tree, state contract, bootstrap steps, decisions, requirements, and sources |

# 1. Document Control and Normative Language

## 1.1 Document control

| Control                   | Rule                                                                                                                  |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Identifier                | SPEC-ELTS-ARCH-001                                                                                                    |
| Version                   | 1.0.0                                                                                                                 |
| Status                    | Implementation baseline                                                                                               |
| Change control            | Change through a pull request. Architecture changes require an ADR and synchronized requirement/workbook updates.     |
| Canonical repository path | docs/architecture/ELTS_ARCHITECTURE_SPECIFICATION.docx, with a generated diffable text export beside it.              |
| Baseline preservation     | Preserve this original DOCX and the workbook byte-for-byte under project-management/baseline/ with SHA-256 checksums. |
| Review cadence            | At each phase gate and before every study release.                                                                    |

## 1.2 Normative words

**MUST / MUST NOT.** An unconditional implementation, safety, validity, or governance requirement. A deviation requires an accepted ADR and, where applicable, investigator or safety approval.

**SHOULD / SHOULD NOT.** The expected design. A deviation is allowed only when its reason, risk, and verification are recorded.

**MAY.** An optional capability that does not weaken any higher-level requirement.

**Evidence.** A resolvable commit, automated test report, checksum, approved document, measurement, photograph, or signed lab record that directly proves a criterion.

**Gate.** A hard, human-approved transition between phases. A gate is never inferred merely because its tasks are complete.

## 2. Purpose, Governing Principle, and Baseline

### 2.1 Governing principle

**Canonical principle:** GitHub contains every non-sensitive artifact required to understand, provision, build, test, review, and reproduce the software. A tagged commit identifies the source, configuration schemas, approved study configuration, documentation, tests, provenance, and build procedure used for a study release.

Portability has two intentionally different forms. Source portability means a developer can clone the repository, run one bootstrap command, validate the exact toolchain, and build or test. Runtime portability means a study operator can copy a prepared release bundle to a compatible Windows PC, extract it anywhere, run FIRST-RUN.cmd and CHECK-SYSTEM.cmd, and launch the study without the Unity Editor.

### 2.2 Baseline observation

On 4 September 2026, the linked public GitHub repository existed and its web interface reported that it was empty. This is an observation, not a permanent assumption. P0.3.S001 requires the implementing agent to inspect the repository again, preserve any work that now exists, and reconcile this specification with current reality before creating files.

The ELTS hardware is described by the source build plan as operational and autonomous, but remote start/stop is not yet wired. The Unity study application, calibration tooling, remote-start link, validation evidence, operator procedures, and analysis ingest do not yet exist in the plan baseline.

### 2.3 Success definition

- A clean clone can be provisioned, validated, tested, and built using documented commands and pinned inputs.
- A copied study release runs from an arbitrary writable path, including a path containing spaces, without the Unity Editor and preferably without network access.
- A Jetson can be provisioned idempotently and diagnosed with a versioned report while preserving local calibration and safety settings.
- An analysis machine can install the locked environment and ingest versioned synthetic or study data without modifying raw inputs.
- Humans and multiple coding agents can determine the exact current phase, gate, task, atomic step, owner, branch, blockers, and evidence from repository files alone.
- Every release is provenance-traceable. Bit-for-bit determinism is not claimed unless separately demonstrated.

## 3. Scope and Non-goals

### 3.1 In scope

- One monorepo for Unity, OpenVR integration, Jetson ELTS integration, analysis, schemas, configuration, scripts, documentation, governance, and progress state.
- Developer, study-PC, Jetson, analysis, and archival portability profiles.
- GitHub rules, branch/PR flow, releases, tags, checks, and beginner-safe multi-machine practice.
- Model-agnostic instructions, thin model adapters, module contracts, ADRs, handoffs, and evidence-based work completion.

- Machine-readable task catalog, append-only progress events, deterministic reducer, and generated PROJECT\_STATE.json.
- Scientific validity, participant/operator separation, data provenance, physical fail-safe boundaries, and release freeze controls.

### 3.2 Non-goals

- This specification does not decide D-01 through D-18. It defines when and how those decisions become accepted ADRs.
- It does not redesign the ELTS vision, servo, or LED-control algorithms. Unity sends lifecycle commands only.
- It does not place participant data, calibration records, credentials, or private deployment overlays in a public repository.
- It does not require containers on the Windows study PC or claim Unity produces byte-identical builds across machines.
- It does not permit an automated test or AI agent to replace physical bench evidence or required human approval.
- It does not add generalized plug-in systems beyond the defined study-v1.0 extension points.

## 4. Scientific and Safety Invariants

These invariants are protected architecture. Tests, code review, schemas, operator checks, and gates must make violations difficult and visible. An agent must treat an apparent need to violate one as a blocker, not an implementation choice.

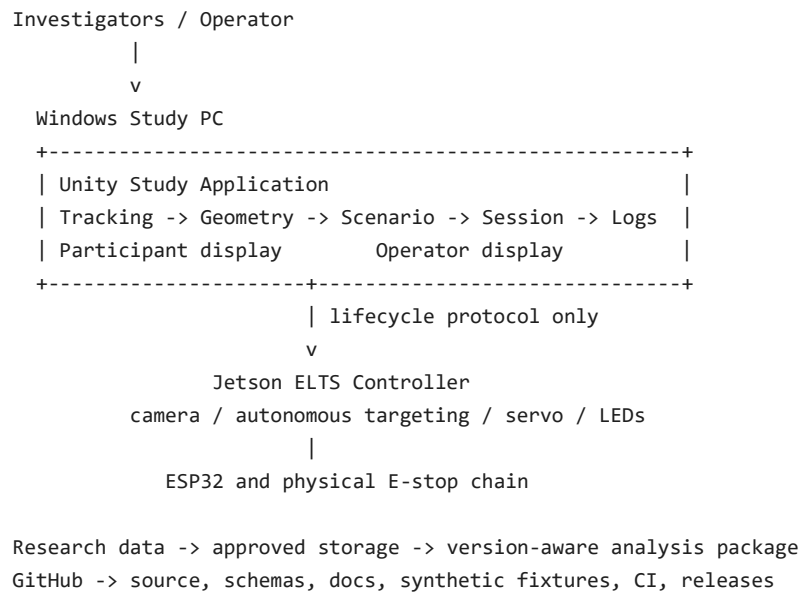
| ID       | Invariant              | Normative statement                                                                                              |
|----------|------------------------|------------------------------------------------------------------------------------------------------------------|
| SCI-001  | Aim error              | World-space angular error in degrees between the zero-corrected bore ray and the muzzle-to-target ray.           |
| SCI-002  | Target state           | Target world position is canonical; screen coordinates are derived each frame.                                   |
| SCI-003  | ELTS independence      | Unity sends lifecycle commands only; never poses, target coordinates, turret angles, or individual LED commands. |
| SCI-004  | No HMD                 | No head-mounted display and nothing near the eyes beyond approved measurement equipment.                         |
| SCI-005  | One clock              | One monotonic application clock; UTC is an alignment anchor only.                                                |
| SCI-006  | Validity               | Every sample carries connection and validity flags; invalid poses are excluded and never interpolated.           |
| SCI-007  | Binding                | Trackers bind by configured serial number, never device index.                                                   |
| SCI-008  | Prediction             | Logged data uses zero prediction; rendering-only head prediction is isolated; weapon remains raw.                |
| SCI-009  | Trigger                | Accept first falling edge and apply a configured lockout, nominally 20 ms.                                       |
| SCI-010  | Primary DV             | Count TargetDestroyed events in each fixed 300-second block.                                                     |
| SCI-011  | Modularity             | Scenario, behavior, scoring, session, geometry, display, and transport choices are data- or interface-driven.    |
| SCI-012  | Display isolation      | Participant and operator render paths are independent; operator content never reaches the participant display.   |
| SAFE-001 | Independent safeguards | Heartbeat timeout, maximum on-time, and normally closed physical E-stop operate independently.                   |
| SAFE-002 | Physical authority     | Software cannot override or weaken the physical E-stop.                                                          |
| SAFE-003 | Change control         | Safety-critical ELTS changes require human review and repeated fail-safe and bench testing.                      |

## 4.1 Enforcement pattern

- Give each invariant a stable ID and link it from code, module contracts, requirements, tests, pull requests, and gate evidence.
- Protect core math and state transitions with deterministic unit tests and synthetic fixtures.
- Protect display and timing behavior with integration tests and logged diagnostics.
- Protect hardware/safety behavior with approved bench procedures and human signoff; CI cannot substitute for these records.
- Record any change to an invariant as an architecture and protocol impact requiring investigator approval.

## 5. System Context and Component Boundaries

### 5.1 Context



### 5.2 Ownership boundaries

| Component              | Owns                                                                                                                                      | Must not own                                                                                               |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Unity application      | Study lifecycle, OpenVR sampling, geometry, rendering, targets, scoring, calibration UI, operator UI, session logs, ELTS lifecycle client | ELTS aiming, turret control, individual LEDs, participant-private storage outside its approved output root |
| Jetson ELTS service    | Transport listener, state machine, command validation, acknowledgements, heartbeat, local event log, safety guards                        | Study target coordinates, head or weapon poses from Unity, scientific aim-error computation                |
| ESP32 / hardware chain | Existing actuation plus normally closed physical E-stop authority                                                                         | Trust in Unity connectivity or UI state for safety                                                         |
| Analysis package       | Immutable ingest, schema/version detection, validation, derived datasets, reports                                                         | Modification or silent repair of raw files                                                                 |
| Progress tooling       | Plan import, events, validation, exact-current-state reduction, evidence references                                                       | Scientific runtime behavior or external tracker truth                                                      |
| GitHub                 | Source collaboration, review, checks, tags, non-sensitive releases                                                                        | Participant data, private hardware credentials, raw lab records                                            |

## 5.3 Runtime startup order

1. Start the one monotonic clock and record the UTC anchor.
2. Load schemas and all allowed configuration layers; reject unknown, missing, or mistyped required fields; compute hashes.
3. Enumerate displays, require the configured participant/operator separation, and present identification output.
4. Initialize OpenVR in background mode after SteamVR is available; report precise remediation when unavailable.
5. Bind trackers by serial number and wait for valid poses within the configured timeout.
6. Start the tracking producer and bounded logging pipeline.
7. When a session contains a WE block, connect to the ELTS service and exchange HELLO/version information.
8. Expose the complete self-check on the operator display. Enable session setup only when mandatory checks pass.

## 6. Canonical Monorepo Architecture

The monorepo is the single collaboration unit. It avoids cross-repository version drift and lets one commit describe the Unity client, Jetson protocol, analysis reader, schemas, tests, documentation, and release procedure together.

### 6.1 Top-level roles

| Path                                        | Role                                                                                  | Change trigger                                          |
|---------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------|
| README.md                                   | Human entry point and quick start                                                     | Any user-visible bootstrap or release-flow change       |
| AGENTS.md                                   | Canonical model-agnostic coding-agent entry point                                     | Agent policy or authority change                        |
| CLAUDE.md / .github/copilot-instructions.md | Thin pointers and tool-specific mechanics                                             | Adapter mechanics only; never duplicate policy          |
| PROJECT_STATE.json                          | Generated exact current project snapshot                                              | Reducer output after accepted progress events           |
| unity/                                      | Unity 6 application, project settings, tests, vendored OpenVR                         | Unity/runtime work                                      |
| jetson/                                     | ELTS listener, protocol adapters, service, installer, tests                           | ELTS-link work                                          |
| analysis/                                   | Locked, version-aware ingest and synthetic tests                                      | Schema or analysis work                                 |
| config/                                     | Only hand-edited configuration source, schemas/examples/local templates               | Study, rig, scenario, session, or local-template change |
| schemas/                                    | Versioned runtime, log, protocol, build, diagnostic, and progress schemas             | Contract change                                         |
| scripts/                                    | Stable one-command developer, build, test, doctor, package, and study-PC entry points | Workflow change                                         |
| tools/progress/                             | Catalog importer, event writer, reducer, validator, report                            | Progress contract change                                |
| project-management/                         | Immutable baselines, task catalog, events, evidence indexes, change log               | Plan or progress change                                 |
| docs/                                       | Architecture, ADRs, module contracts, agent protocol, operator and validation docs    | Behavior or decision change                             |
| .github/                                    | PR templates, issue forms, CODEOWNERS, workflows, dependency policy                   | Collaboration or CI change                              |

## 6.2 Repository policies

- Paths are resolved relative to the repository root or release root; committed user-specific absolute paths are forbidden.
- No Git submodules before study-v1.0. The OpenVR integration is vendored with its version, license, origin, and checksums.
- Commit Unity ProjectSettings, Packages/manifest.json, Packages/packages-lock.json, and every .meta file.
- Ignore Unity Library, Temp, Logs, obj, Build, Builds, and UserSettings directories.
- Do not commit generated study data, participant calibration, local.json, credentials, hardware secrets, or private deployment overlays.
- Binary architecture/workbook baselines receive normalized, diffable exports and a reconciliation test; the originals remain unchanged.
- Large binary assets require an explicit LFS decision. Do not add Git LFS by habit; record why, retention implications, and release behavior.

## 7. Portability and Provisioning

### 7.1 Portability profiles

| Profile              | Entry experience                                           | Required outcome                                                                                     |
|----------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Developer PC         | git clone, then scripts/bootstrap-dev.ps1                  | Pinned prerequisites detected; local configuration generated; tests and build entry points available |
| Study PC             | Copy release ZIP, extract anywhere, run FIRST-RUN.cmd      | No Unity Editor; machine checks and local settings complete; offline-capable study launch            |
| Jetson               | Copy/clone package, run installer and doctor               | Versioned service installed; local config preserved; startup/service/heartbeat tests pass            |
| Analysis workstation | Clone/copy analysis package, run one setup command         | Exact Python environment installed; synthetic ingest passes; raw inputs remain immutable             |
| Archive/recovery     | Retrieve tagged source plus release artifact and manifests | Provenance, checksums, docs, and compatible setup instructions reconstruct a usable release          |

### 7.2 Provisioning contract

- Every setup command is idempotent: a second successful run makes no harmful change and reports already-satisfied checks.
- Before replacing a local configuration or calibration file, back it up using a timestamped, user-visible path.
- Scripts locate their own repository or release root and work from any current directory and any writable extraction path, including paths with spaces.
- Unbundled prerequisites are declared with supported versions and detected before changes are attempted.
- Operations fail closed with a precise error, remediation, non-zero exit code, and a versioned JSON report.
- A copied/extracted release is re-verified from its destination using MANIFEST.sha256 before launch.
- A new physical rig receives a new rig ID and required calibration. The software never silently treats copied calibration as valid for different hardware.

### 7.3 Same machine, replacement machine, and new rig

| Case | Portable items | Must be regenerated or verified |
|------|----------------|---------------------------------|
|------|----------------|---------------------------------|

| Case                         | Portable items                                                      | Must be regenerated or verified                                                                                            |
|------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Same rig, rebuilt study PC   | Tagged release, approved study config, non-sensitive rig descriptor | local.json, display indices, prerequisite checks, release checksums, daily registration                                    |
| Replacement PC with same rig | As above                                                            | Performance/driver baseline, display mapping, OpenVR/SteamVR checks, copied calibration checksum and physical verification |
| New physical rig             | Source, schemas, scenario/session configuration                     | Rig ID, tracker serial mapping, pivot, screen corners, weapon zero, eye offset, validation gates                           |
| Developer laptop             | Repository source and synthetic fixtures                            | Unity/SDK installation, local paths; no assumption that lab hardware exists                                                |

## 8. Toolchain, Dependency Locking, and Reproducibility

### 8.1 Toolchain manifest

The initial agent must create a single machine-readable toolchain manifest and make scripts read it. Human-readable documentation may summarize the values but must not become a second editable source.

| Tool/dependency  | Pin or record                                                             | Enforcement                                                                         |
|------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Unity            | Exact supported Unity 6 LTS patch; do not invent a patch version          | ProjectVersion.txt, optional .unity-version, bootstrap and doctor checks, build log |
| Unity packages   | manifest.json and packages-lock.json                                      | Lockfile CI and clean restore                                                       |
| OpenVR           | Upstream revision/release, C# binding, native DLL hash, license           | Third-party manifest and checksum audit                                             |
| Python           | Supported interpreter range plus pyproject.toml and uv.lock               | Locked sync, tests, analysis provenance                                             |
| Jetson           | Hardware model, OS image, architecture, Python/service dependencies       | Documented baseline and doctor report; decide exact values from real device         |
| Windows          | Supported edition/build family, GPU and driver evidence, SteamVR settings | Doctor report and validation baseline                                               |
| Repository tools | Schema/progress tool versions                                             | Self-reported version and deterministic fixtures                                    |

### 8.2 Reproducibility contract

- A clean Git commit plus recorded toolchain inputs must produce a functionally equivalent release and the same validated schemas/configuration.
- The release build runs from a clean worktree. A developer build may be dirty only when its UI and build-info label it as non-study and record the diff state.
- The build records commit, tag, tool versions, target, package locks, schema versions, configuration hashes, build time, and artifact checksum.
- Build scripts pass an explicit Unity project path and build target/profile and use batch-mode logging suitable for CI.
- If byte-identical builds are later required, create a separate ADR and reproducibility test; do not imply that provenance equivalence is byte equality.

## 9. Configuration and Schema Architecture

### 9.1 Single source and generated staging

```

config/           # sole hand-edited configuration namespace
schemas/         # or canonical links into schemas/config/
defaults/        # safe, explicit defaults

```

```

study/                # versioned study/session/scenario files
rig/templates/        # public templates; no real secret/participant data
local.example.json    # committed whitelist template
local/                # generated or deployment-local; ignored

scripts/stage-config.ps1
  validates + hashes + copies approved effective config
    -> unity/Assets/StreamingAssets/config-generated/ # generated, never hand-edited
    -> release/config/                               # frozen release copy

```

Root config is the only hand-edited source. Unity StreamingAssets receives a staged, generated copy. CI regenerates it and fails if a committed generated output is stale or if two hand-edited sources diverge.

## 9.2 Layers and precedence

| Order | Layer              | Disposition                                               | Examples                                                                    |
|-------|--------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------|
| 1     | Schema defaults    | Committed, versioned                                      | Safe defaults that do not decide unresolved research values                 |
| 2     | Study config       | Committed and frozen by release hash                      | Scenarios, sessions, condition structure, approved thresholds               |
| 3     | Rig profile        | Committed template or approved private deployment overlay | Rig identity, nominal geometry, tracker serial placeholders                 |
| 4     | Calibration        | Generated, protected local/research storage               | Pivot, corners, weapon zero, eye offset; never silently reused on a new rig |
| 5     | Machine local      | Generated/untracked local.json                            | Display indices, data root, device ports; whitelist only                    |
| 6     | Session assignment | Research-data location                                    | Participant pseudonym, block order, run suffix                              |

## 9.3 Schema rules

- Every configuration type has a versioned JSON Schema with strict additional-property handling where practical.
- Missing, unknown, out-of-range, or mistyped required fields produce a precise startup failure and no partial session.
- A schema change increments the schema version and supplies a migration or declares incompatibility.
- All effective configuration files are canonicalized, SHA-256 hashed, displayed to the operator, and recorded in logs/build provenance.
- During data collection, a frozen study-config change requires a new Git tag and study-log entry. local.json is excluded from the study freeze but still recorded by hash.

# 10. Unity Application Architecture

## 10.1 Module boundaries

| Module   | Owns                                                                    | Critical boundary                                      |
|----------|-------------------------------------------------------------------------|--------------------------------------------------------|
| Clock    | Monotonic timestamps, UTC anchor                                        | Thread-safe read-only time API                         |
| Config   | Load, validate, resolve, hash, expose immutable typed config            | No module reads arbitrary JSON directly                |
| Tracking | OpenVR lifecycle, serial binding, 250 Hz sampling, validity, conversion | Dedicated producer; zero prediction for logged samples |
| Geometry | Bore ray, screen basis/projection, angular error, visibility            | Pure deterministic math where possible                 |
| Logging  | Bounded queues, samples, targets, events, session summary               | Writer thread owns files; no Unity API calls           |

| Module      | Owns                                                           | Critical boundary                                             |
|-------------|----------------------------------------------------------------|---------------------------------------------------------------|
| Rendering   | Off-axis participant camera and independent operator view      | Main thread; operator layers excluded from participant camera |
| Scenario    | Spawn, movement, shot, scoring strategies                      | Interfaces and config select implementations                  |
| Session     | Setup, calibration, practice, randomized blocks, breaks, close | Explicit state machine, idempotent abort/close                |
| Calibration | Pivot, corner probe, weapon zero, eye offset, 3x3 verification | Wizard workflows with residuals and provenance                |
| ELTS        | Lifecycle client, transport adapter, ack/time-offset handling  | Never contains ELTS aim/servo/LED logic                       |
| Operator    | Status, self-check, control, warnings, incident markers        | Never rendered on participant display                         |

## 10.2 Thread ownership

| Execution context | Allowed work                                                                             | Prohibited work                                                              |
|-------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Unity main thread | Scene objects, rendering, UI, session transitions, consumption of immutable snapshots    | Blocking I/O, high-frequency polling loops                                   |
| Tracking producer | OpenVR pose/event polling, conversion, sequence numbering, lock-free/bounded publication | UnityEngine object access, file I/O                                          |
| Logging writer    | File creation, serialization, flush/fsync policy, checksums                              | Unity API calls, scientific state mutation                                   |
| ELTS I/O worker   | Transport read/write, framing, retries, acknowledgement timing                           | Direct LED authority or session-state mutation without queued command result |

## 10.3 Coordinate and timing rules

- The room frame is the selected SteamVR tracking universe converted once into Unity left-handed +X right, +Y up, +Z forward, meters.
- Handedness conversion exists in exactly one Tracking function. Downstream code never reasons in OpenVR-native coordinates.
- Tracker-local offsets and directions use Unity convention after conversion. Logged poses are room-frame, Unity convention, before calibration corrections.
- World target position is canonical. Projection to participant-display u/v happens from current eye and display geometry each frame.
- All sample/event timestamps use the one monotonic clock. UTC, Jetson timestamps, and collaborator markers are alignment metadata.

## 10.4 Tests

- EditMode: coordinate conversion, projection, angular error, schemas, configuration precedence, randomization, scoring, reducer-independent state machines.
- PlayMode: display isolation, startup failures, session transitions, synthetic tracking, calibration wizards, rendering geometry, orderly abort and close.
- Hardware integration: OpenVR enumeration/reconnect, sample rate/jitter, trigger edge behavior, display mapping, ELTS transport, failure injection.
- Soak: 30-minute full-load run with sequence/drop counters, frame hitches, disk behavior, and structured evidence.

## 11. ELTS Jetson Integration

### 11.1 Transport-independent protocol

D-03 selects the transport, but every candidate carries newline-delimited JSON with a versioned message schema. Unity depends on IEltLink, not on serial, UDP, or another concrete mechanism.

Idle --HELLO/ARM--> Armed --START--> Active --STOP/timeout/guard/E-stop--> Armed --DISARM--> Idle

| State  | Permitted behavior                                  | Entry/exit rules                                                   |
|--------|-----------------------------------------------------|--------------------------------------------------------------------|
| Idle   | Camera/tracking loop may be off; LEDs inhibited     | Startup and DISARM destination                                     |
| Armed  | Pipeline may run; turrets may track; LEDs inhibited | ARM after version/config checks; D-10 decides use during NE blocks |
| Active | LEDs permitted under ELTS-owned control             | START only; STOP, timeout, max-on-time, guard, or E-stop exits     |

### 11.2 Message contract

| Direction       | Message        | Required content/behavior                                                         |
|-----------------|----------------|-----------------------------------------------------------------------------------|
| Unity to Jetson | HELLO          | Sequence, application version, protocol version; reject incompatible versions     |
| Unity to Jetson | ARM / DISARM   | Sequence and Unity monotonic time                                                 |
| Unity to Jetson | START          | Sequence, Unity time, participant pseudonym, block, condition, duration           |
| Unity to Jetson | STOP           | Sequence, Unity time, reason: block_end, abort, or operator                       |
| Unity to Jetson | PING / STATUS? | Heartbeat timing or status query                                                  |
| Jetson to Unity | ACK / NACK     | Same sequence, command, Jetson time, state, and error for NACK                    |
| Jetson to Unity | STATE          | Unsolicited state change: time, state, LED permission, face-detected flag, reason |

Every command receives exactly one ACK or NACK with the same sequence. Unknown messages are NACKed. ACK to START is emitted only when LEDs become permitted. Default design values are one-second PING and three-second heartbeat timeout, but configuration and both endpoints must agree; safety approval controls final limits.

### 11.3 Failure behavior

- Loss of heartbeat, maximum on-time, guard failure, process crash, transport failure, or physical E-stop independently removes LED permission.
- Reconnect never resumes Active automatically. Unity and Jetson re-establish HELLO, version, state, and operator-visible readiness.
- Commands are idempotent by sequence and state. Duplicate messages cannot extend maximum on-time or create a second transition.
- Both ends log sequence, send/receive monotonic times, acknowledgements, offset estimates, state changes, retries, and reasons.
- Physical E-stop authority remains outside software. Software reports it but cannot mask, override, or remotely reset it.

## 12. Logging, Research Data, and Analysis

### 12.1 Log products

| Product          | Purpose                                                                                      | Core rules                                                                                                     |
|------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| samples          | Approximately 250 Hz synchronized tracking and derived aim data                              | Sequence numbers, monotonic time, raw converted poses, validity flags, zero-prediction basis, no interpolation |
| targets          | Per-block target lifecycle and parameters                                                    | World position canonical; IDs and scenario seed/version recorded                                               |
| events           | Session, trigger, shot, hit/destroy, calibration, display, ELTS, sync, warning, abort events | One clock, structured type/payload, provenance                                                                 |
| session summary  | Human and machine-readable closure record                                                    | Config hashes, build identity, block outcomes, warnings, file checksums                                        |
| Jetson event log | Independent ELTS state evidence                                                              | Sequence, Jetson clock, Unity alignment values, LED permission, failure reason                                 |

### 12.2 Write and recovery rules

- The logging writer is the sole owner of open output files and uses bounded queues so disk work cannot block tracking.
- Files are created with create-new semantics. A rerun uses a unique suffix; raw output is never overwritten or edited in place.
- The plan baseline calls for short periodic flushes and a durable block-end flush; exact filesystem behavior is measured and documented.
- Low disk, dropped samples, serialization errors, and late events become operator-visible warnings and structured events.
- Session close writes checksums and validates the complete output set. Crash recovery preserves partial data and marks it incomplete.

### 12.3 Data governance

- Participant names, contacts, consent, pseudonymous IDs, calibration, and study data never enter Git or public GitHub.
- Only intentionally synthetic fixtures approved for public disclosure may be committed.
- Hardware serials, network details, and calibration may be sensitive. Commit templates and schemas; use an approved private deployment overlay where required.
- Prepared study operation sends no telemetry or research data to unapproved cloud services and should not require internet access.
- Backups include per-session checksums and follow the approved IRB and institutional retention plan. This repository records procedures, not participant records.

### 12.4 Analysis contract

- Ingest detects log schema and application versions, validates all required files/hashes, and rejects unsupported versions clearly.
- Raw inputs are opened read-only. Any migration writes a new derived artifact and records source hashes, tool version, and migration chain.
- Analysis explicitly excludes invalid tracking samples; it never interpolates them into the primary DV.
- Synthetic fixtures cover valid sessions, dropouts, corrupted rows, version changes, partial sessions, duplicate target IDs, and unknown fields.

- The first study release requires successful end-to-end ingest of at least three complete pilot sessions.

## 13. Build, CI, Diagnostics, and Release

### 13.1 Stable commands

| Entry point               | User outcome                                                                                     |
|---------------------------|--------------------------------------------------------------------------------------------------|
| scripts/bootstrap-dev.ps1 | Validate/install allowed developer prerequisites, generate local config, print next action       |
| scripts/doctor.ps1        | Readable diagnostics plus versioned doctor-report.json; no mutation unless explicitly requested  |
| scripts/test.ps1          | Run progress/schema/policy/Python/Unity test groups that are available on this machine           |
| scripts/build.ps1         | Invoke the pinned Unity Editor with explicit project path and target/profile; produce build-info |
| scripts/package-study.ps1 | Assemble a self-contained study bundle and checksum manifest from a clean tagged commit          |
| FIRST-RUN.cmd             | Study-PC provisioning wrapper; idempotent and operator-readable                                  |
| CHECK-SYSTEM.cmd          | Read-only preflight with machine-readable report                                                 |
| START-ELTS.cmd            | Launch the released operator workflow only after successful checks                               |

### 13.2 CI groups

| Group                  | Checks                                                                                                        | Required on PR                                                             |
|------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Repository policy      | Forbidden paths/files, secret/data scan, absolute paths, Unity ignore/meta rules, third-party notices         | Yes                                                                        |
| Schemas/config         | JSON Schema validation, examples, migrations, staging regeneration, hashes                                    | Yes                                                                        |
| Progress               | Catalog/event schemas, legal transitions, deterministic reduction, state regeneration diff, ID reconciliation | Yes                                                                        |
| Python/Jetson/analysis | Formatting/lint as selected, unit tests, synthetic protocol and ingest fixtures                               | When affected; baseline smoke always                                       |
| Unity EditMode         | Pure math, config, state, logging, invariant tests                                                            | When Unity runner/license available; otherwise required pre-merge evidence |
| Unity PlayMode/build   | Startup/display/session/calibration synthetic path and Windows build                                          | Required before study-release branch/tag                                   |
| Docs/contracts         | Broken links, required headings, ADR/module metadata, behavior-doc parity checklist                           | Yes                                                                        |
| Portability            | Clean clone, path with spaces, repeated bootstrap, copied release, offline smoke where feasible               | Nightly/release and after provisioning changes                             |

Hardware gates remain external evidence. CI verifies that required evidence references and approvals exist; it does not fabricate physical test results.

### 13.3 Release bundle

```

ELTS-study-vX.Y.Z/
  FIRST-RUN.cmd
  CHECK-SYSTEM.cmd
  START-ELTS.cmd
  app/
  config/
  # Windows standalone build
  # frozen approved config + local template

```

```

operator-docs/
licenses/
schemas/
build-info.json
MANIFEST.sha256
RELEASE_NOTES.md

```

- Build from a clean tagged commit. A dirty build is visibly non-study and cannot collect participant data.
- Verify the archive, copy it to a new destination with spaces in its path, extract, verify again, run FIRST-RUN twice, run CHECK-SYSTEM, and smoke-launch.
- The first data-collection release is study-v1.0 only after G8 passes and a human release approver signs off.
- Any behavior/config/schema change during data collection creates a new tag and study-log entry with impact and compatibility notes.

## 14. GitHub and Multi-Machine Workflow for a Beginner

### 14.1 Mental model

**Repository.** The project and its complete history.

**Clone.** A full working copy on one machine. Each machine has its own clone and local branches.

**Commit.** A named snapshot of changes in a local clone.

**Branch.** A temporary line of work. ELTS uses one branch per task and one active owner.

**Push.** Send local commits to GitHub.

**Pull/fetch.** Learn about and incorporate remote work into the local clone.

**Pull request.** A review proposal to merge a task branch into main after checks and human review.

**Tag/release.** An immutable study-version marker and its packaged artifacts.

### 14.2 Normal task flow

1. On the machine that will own the task, confirm no other machine or agent owns the same task/branch.
2. Fetch GitHub, switch to main, and update it without creating a merge commit.
3. Read AGENTS.md, PROJECT\_STATE.json, the assigned workbook row, relevant module contract, requirements, decisions, and evidence needs.
4. Create one short-lived branch named from the task, for example feat/P2.4-geometry-math.
5. Record a task\_started progress event with owner, branch, atomic step, claimed paths, and expected checks; regenerate PROJECT\_STATE.json.
6. Make small coherent commits. Pull/fetch regularly and keep scope limited to the task.
7. Run required checks; record results and evidence. Update code, tests, docs, schemas, and module contracts together.
8. Push the branch and open a pull request linked to task/requirements/decisions. Resolve comments deliberately.
9. After checks and required human approval, squash-merge into main and delete the remote branch.
10. On every other machine, finish or stash unrelated local work, switch to main, and pull the new history before starting another task.

## 14.3 Multi-machine rules

- GitHub is the rendezvous point. Do not copy an active repository folder between machines as a substitute for push/pull.
- Never edit the same task branch concurrently on two machines. If work must move, push it, record a handoff, stop on the old machine, then continue on the new machine.
- Parallel work uses separate branches and preferably separate Git worktrees with non-overlapping modules/claimed paths.
- A merge conflict means two histories touched related content. Understand both intents, resolve the file, rerun affected checks, and document the resolution.
- Do not force-push or delete main. Do not rewrite shared task history unless a maintainer explicitly coordinates it.

## 14.4 GitHub settings

- Use main as the only permanent integration branch. Require pull requests, status checks, resolved conversations, and a linear history.
- Use squash merge by default and delete merged branches. Require CODEOWNER/research/safety review for protected paths.
- Block force pushes and branch deletion. Require signed commits only if the team can support it consistently; record the decision.
- Apply rules using a repository ruleset compatible with the account/repository plan. Verify the effective protection rather than assuming configuration succeeded.
- GitHub Issues and Trello may mirror work for convenience. Stable IDs and repository progress files remain authoritative when systems disagree.

# 15. Documentation and ADR Governance

## 15.1 Canonical documentation chain

README.md -> AGENTS.md -> PROJECT\_STATE.json -> task row -> module contract -> ADRs/schemas/tests

- README.md tells a human how to start and links to authoritative details.
- AGENTS.md is the model-agnostic agent entry point. CLAUDE.md and other adapters point to it and contain only tool-specific mechanics.
- docs/ai/context-index.md maps task categories to the smallest required context set so agents do not load the entire repository blindly.
- Module contracts make boundaries, thread ownership, configuration, events, failure behavior, tests, invariants, ADRs, and work-item links explicit.
- Documentation describing changed behavior is updated in the same pull request; a later documentation task is not an acceptable default.

## 15.2 Module contract template

| Required heading          | Question answered                                 |
|---------------------------|---------------------------------------------------|
| Purpose and owner         | Why does the module exist and who reviews it?     |
| Boundaries                | What does it own and explicitly not own?          |
| Public interfaces         | What APIs/messages/files are stable?              |
| Thread/process ownership  | Where may each operation run?                     |
| Configuration and schemas | What values are accepted, layered, and versioned? |

| Required heading            | Question answered                                             |
|-----------------------------|---------------------------------------------------------------|
| Inputs, outputs, and events | What data enters/leaves and how is it timestamped?            |
| Failure behavior            | How does it fail closed and how does an operator recover?     |
| Tests and evidence          | What proves correctness?                                      |
| Invariants and ADRs         | Which protected rules and accepted decisions constrain it?    |
| Work-item links             | Which P, atomic-step, gate, and requirement IDs implement it? |

### 15.3 ADR lifecycle

- ADRs are append-only historical records with Proposed, Accepted, Rejected, or Superseded status.
- Never renumber D-01 through D-18. An ADR records one or more decision IDs and keeps their traceability.
- An accepted ADR states context, decision, alternatives, consequences, requirements affected, migration, verification, approvers, and date.
- A superseding ADR links both directions. Do not edit history to make the old decision appear never to have existed.
- Research/safety decisions require the named human authority. An agent may prepare options and evidence but cannot self-approve them.

## 16. Multi-LLM and Multi-Agent Contribution Protocol

### 16.1 Before editing

1. Read AGENTS.md and the authority hierarchy; identify protected scientific/safety requirements.
2. Validate PROJECT\_STATE.json by rerunning the reducer. If it differs, stop and reconcile before trusting it.
3. Read the assigned task and atomic-step rows, dependencies, decisions, gates, requirements, artifacts, and required evidence in the workbook/task catalog.
4. Inspect the current repository, uncommitted changes, active task claims, and branch ownership. Preserve user work.
5. Declare actor, task, atomic step, branch, claimed paths, scope, expected checks, and stop conditions in a task\_started event.

### 16.2 During work

- Stay inside the declared task. Do not perform unrelated cleanup or reinterpret open decisions as permission to invent values.
- Prefer interfaces and configuration for expected variability; keep defaults and adjustable values centralized and documented.
- Update tests and documentation in the same change as an interface, schema, config, or observable behavior change.
- Record meaningful verification commands/results and repository-relative evidence. Never record secrets, participant data, or user-specific absolute paths.
- If an invariant, authority, dependency, or unresolved decision blocks work, emit a blocked event with the exact blocker ID and required human action.
- Do not use destructive Git operations, overwrite another agent's work, merge/release without authority, or mark done without evidence.

## 16.3 Completion and handoff

1. Run all checks named by the task and any additional checks required by the affected contracts.
2. Record a `verification_pending` event and attach resolvable evidence; use done only after every acceptance criterion is proven.
3. Regenerate `PROJECT_STATE.json` and machine-readable summaries; verify a clean second reduction produces no diff.
4. Write a handoff containing completed work, remaining work, files changed, checks/results, evidence, risks, blockers, next eligible step, and branch/commit.
5. Open or update the pull request. An agent never represents its own unreviewed output as human approval.

## 16.4 Parallel work and conflict control

- Only one active owner may claim a task/branch. Two `task_started` events from the same task revision are a reducer error.
- Parallel agents use separate worktrees/branches and disjoint claimed paths. Shared contract files need an explicit integration owner.
- Task events are separate append-only files to minimize merge conflicts; event chains detect competing transitions.
- A generated-state conflict is never resolved by choosing one `PROJECT_STATE.json`. Merge the valid events, rerun the reducer, and commit its result.
- A handoff transfers ownership only after the prior owner records a release/handoff event and stops editing.

# 17. Machine-Readable Plan and Progress-State System

## 17.1 Required files

| Path                                                       | Authority                                                                                       | Mutation rule                                                           |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <code>project-management/baseline/*</code>                 | Original specification/workbook and SHA-256 manifest                                            | Immutable after bootstrap                                               |
| <code>project-management/task-catalog.json</code>          | Normalized phases, tasks, atomic steps, decisions, gates, dependencies, requirements, artifacts | Generated from a versioned plan baseline; IDs never silently renumbered |
| <code>project-management/progress/events/**/* .json</code> | Append-only facts                                                                               | One event per new file; never rewrite accepted history                  |
| <code>schemas/progress/*.schema.json</code>                | Task, event, state, evidence, handoff contracts                                                 | Versioned with migration/compatibility declaration                      |
| <code>tools/progress/*</code>                              | Importer, event command, reducer, validator, human report                                       | Tested deterministic implementation                                     |
| <code>PROJECT_STATE.json</code>                            | Generated exact current snapshot                                                                | Never hand-edit after reducer bootstrap                                 |
| <code>project-management/progress/handoffs/*</code>        | Durable human/agent handoff records                                                             | Append-only and schema-validated                                        |

## 17.2 Event model

Each event is a self-contained JSON object stored under a collision-resistant filename such as `events/2026/09/20260904T180000.000Z_agent-codex_P0.3.S001_<uuid>.json`. The event targets a task, atomic step, decision, gate, or plan change and records its expected prior revision/hash.

```

{
  "$schema": "../../../schemas/progress/progress-event.schema.json",
  "schemaVersion": 1,
  "eventId": "5f31f504-1c45-4e8e-b130-9b4f4cb2d7f1",
  "eventType": "task_started",
  "recordedAtUtc": "2026-09-04T18:00:00Z",
  "actor": {"type": "agent", "id": "codex", "tool": "Codex"},
  "planVersion": "ELTS-build-plan-1.0",
  "target": {"kind": "atomic_step", "id": "P0.3.S001"},
  "taskId": "P0.3",
  "fromStatus": "ready",
  "toStatus": "in_progress",
  "taskRevision": 1,
  "previousTaskEventHash": null,
  "branch": "bootstrap/repository-governance",
  "claimedPaths": ["project-management/", "schemas/progress/", "tools/progress/"],
  "evidence": [],
  "blockers": [],
  "note": "Inventory repository and preserve all existing work."
}

```

### 17.3 Legal task states

| From                                       | To                   | Required condition                                            |
|--------------------------------------------|----------------------|---------------------------------------------------------------|
| not_started                                | ready                | Dependencies and current phase/gate permit work               |
| ready                                      | in_progress          | One owner claims one branch and scope                         |
| in_progress                                | verification_pending | Implementation complete; required checks/evidence underway    |
| verification_pending                       | done                 | Every acceptance criterion has resolvable evidence            |
| verification_pending                       | in_progress          | Verification failed; remediation resumed                      |
| ready / in_progress / verification_pending | blocked              | Exact blocker and next required action recorded               |
| blocked                                    | ready / in_progress  | Blocker resolution recorded                                   |
| done                                       | in_progress          | Explicit reopen reason and human authorization where required |

### 17.4 Deterministic exact-current-step algorithm

1. Choose the earliest phase in plan sequence whose gate has not passed.
2. Within that phase, validate dependencies/decisions and collect all in\_progress atomic steps whose event chains are valid.
3. If active steps exist, current.primaryStepId is the lowest workbook sequence; current.activeStepIds lists all active steps in sequence order.
4. Otherwise choose the lowest-sequence verification\_pending step.
5. Otherwise choose the lowest-sequence ready step and list the other ready steps in current.nextEligibleStepIds.
6. If all otherwise eligible work is blocked, set current.mode to blocked and list exact blocker IDs and required actions.
7. After an explicit evidence-backed G8 pass event, set current.mode to complete and leave current.primaryStepId null.

8. If two events claim the same previous task revision/hash or an illegal transition occurs, fail reduction and require reconciliation; never choose by timestamp alone.

## 17.5 Generated state example

```
{
  "$schema": "schemas/progress/progress-state.schema.json",
  "schemaVersion": 1,
  "project": "DESS-ELTS-Research-Software",
  "planVersion": "ELTS-build-plan-1.0",
  "generatedAtUtc": "2026-09-04T18:00:00Z",
  "generatedFrom": {
    "workbookSha256": "<64 hex>",
    "specificationSha256": "<64 hex>",
    "taskCatalogSha256": "<64 hex>",
    "eventCount": 1
  },
  "current": {
    "mode": "working",
    "phaseId": "P0",
    "gateId": "G0",
    "primaryStepId": "P0.3.S001",
    "activeStepIds": ["P0.3.S001"],
    "nextEligibleStepIds": [],
    "blockingIds": []
  },
  "activeWork": [{
    "owner": "agent:codex",
    "taskId": "P0.3",
    "atomicStepId": "P0.3.S001",
    "branch": "bootstrap/repository-governance"
  }],
  "integrity": {"reducerVersion": "1.0.0", "valid": true}
}
```

## 17.6 Reducer and CI contract

- The same catalog and ordered valid event set always produce semantically identical state. generatedAtUtc may be supplied by the caller or excluded from semantic comparison.
- The reducer validates schemas, IDs, dependencies, state transitions, event-chain hashes/revisions, unique ownership, evidence shape, and gate authority.
- An event is never accepted because PROJECT\_STATE.json says it happened; the direction of truth is catalog plus events to generated state.
- CI regenerates state in a temporary location and fails when it differs from committed PROJECT\_STATE.json or normalized workbook exports.
- The command that writes an event validates first, writes atomically to a new path, regenerates state atomically, and prints the new exact current step.
- The workbook is a preserved baseline and human planning view. Ongoing truth is repository events/state; workbook status cells may be regenerated as a report but are not the concurrency authority.

## 18. Implementation Sequence and Hard Gates

The workbook enumerates 91 P-tasks, 243 atomic steps, 18 decisions, 62 gate criteria, 360 normalized dependency edges, 108 requirements, and 26 risks. The task catalog preserves these IDs and sequences. The phase table below is an orientation summary, not a substitute for the workbook.

| Phase | Goal                 | Tasks | Primary result                                                                                        | Gate |
|-------|----------------------|-------|-------------------------------------------------------------------------------------------------------|------|
| P0    | Foundations          | 7     | Repository/provisioning baseline, hardware/PC readiness, decision ownership, safety docs, room survey | G0   |
| P1    | Tracking online      | 14    | SteamVR headless, OpenVR spikes, serial binding, 250 Hz polling, trigger, fabrication starts          | G1   |
| P2    | Core services        | 9     | Config, clock, tracking service, geometry, logging, synthetic poses, tests                            | G2   |
| P3    | Rendering            | 8     | Off-axis participant view, independent operator view, displays, prediction and performance            | G3   |
| P4    | Scenario and session | 13    | Targets, movement, shooting, scoring, randomization, operator controls, session lifecycle             | G4   |
| P5    | Calibration          | 10    | Pivot, screen corners, weapon zero, eye offset, 3x3 verification, daily checks                        | G5   |
| P6    | ELTS integration     | 13    | Protocol/transport, Unity client, Jetson listener, ack/timing, heartbeat, E-stop                      | G6   |
| P7    | Bench validation     | 9     | IR, washout, occlusion, registration, soak, latency, safety, frozen thresholds                        | G7   |
| P8    | Pilot and freeze     | 8     | Parameters, operator procedures, backups, sync, pilots, ingest, tagged release                        | G8   |

### 18.1 Gate behavior

- G0 through G8 are hard gates. Downstream work may begin only where the dependency graph explicitly allows synthetic or parallel work.
- A gate becomes `verification_pending` only after all criteria have candidate evidence. It becomes passed only through an `authorized_gate_passed` event.
- Completing every task does not automatically pass a gate. A failed criterion produces a failed or blocked gate state and a remediation path.
- Evidence may be in Git/CI or an approved lab record. Sensitive raw measurements remain outside Git; the event stores a durable approved reference and checksum where permitted.
- G8 is the sole Ready for Participant 1 transition. It requires the tagged standalone build, hardware/calibration/display/logging/ELTS criteria, all Phase 7 evidence, pilots, ingest, and closed decisions except informational D-15.

### 18.2 Decision scheduling

Appendix D lists D-01 through D-18. Open decisions remain explicit data, not guessed values. A task that only needs an interface or placeholder may proceed; a task whose behavior depends on the decision blocks at its declared deadline.

## 19. Security, Privacy, and Safety Controls

| Threat/control area             | Required controls                                                                                                        |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Public repository data leak     | Deny patterns and scans; templates only; no participant IDs/data/calibration; review generated artifacts before commit   |
| Secrets and network credentials | Environment/OS secret storage or approved deployment overlay; never in config examples, logs, issues, or progress events |

| Threat/control area                  | Required controls                                                                                                 |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Supply chain                         | Pinned locks, third-party manifest/licenses/checksums, dependency review, minimal runtime dependencies            |
| Unsafe remote activation             | Version/state handshake, explicit ARM/START, acknowledgement, heartbeat, max on-time, independent physical E-stop |
| Unreviewed AI change                 | Protected paths, invariant tests, human review, hardware re-verification, no self-approval                        |
| Release substitution                 | Clean tagged commit, signed/recorded manifest, destination verification, build-info, operator-visible version     |
| Research-data integrity              | Append-only raw output, unique rerun suffixes, checksums, immutable ingest, migration provenance                  |
| Machine compromise/offline operation | Prepared study runtime avoids unapproved cloud services and should work without network connectivity              |

## 19.1 Human-only approvals

- Changes to the research protocol, primary DV, condition timing, safety limits, E-stop behavior, frozen study configuration, or release readiness.
- Acceptance of D-01 through D-18 by their named owners.
- Pass/fail interpretation of physical bench tests and G8 release authorization.
- Any exception that could expose participant data, hardware secrets, or institutionally controlled information.

## 20. Verification and Acceptance

### 20.1 Repository bootstrap acceptance

- Original specification and workbook are preserved with checksums; normalized exports reconcile all expected counts and stable IDs.
- The repository tree, README, AGENTS.md, adapters, schemas, module/ADR templates, scripts, and CI baseline exist and are internally linked.
- PROJECT\_STATE.json is generated from a schema-valid catalog and append-only events; a second reduction is no-op and the exact current step is printed.
- No open decision is silently filled. Existing repository/user work is preserved and incorporated deliberately.
- Clean-clone, path-with-spaces, repeated-bootstrap, schema, progress, policy, synthetic analysis, Unity smoke/test/build checks run as available and produce evidence.

### 20.2 Portability acceptance matrix

| Test             | Procedure                                                              | Pass condition                                                                               |
|------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Clean clone      | Clone to a new directory and run bootstrap/doctor/test/build           | No undocumented manual edits; pinned inputs and output provenance reported                   |
| Path with spaces | Use a writable path containing spaces                                  | All scripts, build, config staging, and release launch resolve correctly                     |
| Idempotency      | Run bootstrap/FIRST-RUN twice                                          | Second run succeeds without destructive changes; backups/reporting are correct               |
| Study copy       | Copy ZIP, verify, extract elsewhere, verify, first-run, doctor, launch | No Unity Editor; correct build/config identity; operator can reach self-check                |
| Offline          | Disconnect network after preparation                                   | Study operation and diagnostics work unless an explicitly approved dependency says otherwise |

| Test             | Procedure                                                                | Pass condition                                                           |
|------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------|
| New rig          | Apply release to different hardware identity                             | System refuses inherited calibration and requires rig setup/verification |
| Jetson recovery  | Install on approved baseline, reinstall, restart service, interrupt link | Config preserved; correct state; LEDs fail safe; report generated        |
| Analysis rebuild | Install locked environment and ingest synthetic fixtures                 | Same validated derived results/provenance without modifying source       |

## 20.3 Coherence acceptance

- Every requirement ID resolves to implementation and verification targets; every task/step references its dependencies and evidence expectations.
- All IDs and counts reconcile among the preserved workbook, task catalog, generated state, and documentation exports.
- Model-specific instruction files contain no duplicated normative policy and link to AGENTS.md.
- Module contracts match public interfaces and schemas. Documentation-link and behavior-doc parity checks pass.
- Trello/Issues differences are reported as mirror drift and never overwrite repository truth automatically.

## 21. Initial Coding-Agent Directive

**Use:** Attach this specification and the companion workbook, then send the short prompt below. The detailed directive in this section is already part of the attachment and is normative.

### 21.1 Short initial prompt

Read the attached ELTS architecture specification and implementation workbook in full. Treat them as the governing architecture and execution baseline. Begin at Initial Coding-Agent Directive section 21 and the workbook Start Here sheet. Inspect and preserve the repository, build the bootstrap/progress architecture, create PROJECT\_STATE.json through the reducer, and continue only as far as evidence and unresolved human decisions allow. Do not invent research, safety, hardware, or toolchain values.

### 21.2 Required first-run behavior

1. Inspect the repository, Git status/history/remotes, current files, and any existing instructions. Do not assume it is still empty and do not overwrite user work.
2. Read both attachments completely. Verify the source-plan hash recorded in this specification when the plan is available. Copy both originals unchanged into project-management/baseline/ and create SHA-256 manifest entries.
3. Create a bootstrap branch from current main. Record P0.3.S001 as the first task claim only after confirming no competing owner/event exists.
4. Create the canonical monorepo skeleton, README, AGENTS.md, thin model adapters, documentation/ADR/module templates, repository policies, and protected-path ownership files.
5. Normalize the workbook into task-catalog.json and diffable CSV/JSON exports. Validate exactly 91 tasks, 243 atomic steps, 18 decisions, 62 gate criteria, 108 requirements, 26 risks, and 360 dependency edges unless a reviewed plan-change record explains a difference.
6. Create versioned progress schemas and a deterministic tested importer/event writer/reducer/validator/report tool. Seed the minimum valid bootstrap event and generate PROJECT\_STATE.json; do not hand-author a fictional completed state.

7. Create centralized toolchain/config manifests and portable script entry points. Select the exact Unity 6 LTS patch only from a real supported installation/team decision and record it; never invent it.
8. Create the Unity project skeleton, lock manifests, vendored OpenVR provenance structure, synthetic tests, CI baseline, and build/doctor scaffolding in the atomic-step order. If a prerequisite or decision is unavailable, record the exact blocker and continue only on catalog-eligible parallel work.
9. Run proportional verification after each atomic step. Store repository-safe evidence, update documents/contracts with behavior, emit progress events, and regenerate state.
10. Stop before any action needing new human authority, private credentials/data, hardware access that is unavailable, a safety decision, a GitHub settings mutation not authorized by the user, or a release/merge approval. Leave a precise handoff and exact current step.

## 21.3 Required first response/handoff format

### Baseline

- repository condition and preserved existing work
- attachment/checksum status and normalized counts

### Current state

- phase / gate / task / atomic step
- owner / branch / claimed paths
- open blockers and decisions

### Changes

- repository-relative files created or changed
- architecture decisions made versus deferred

### Verification

- command or method, result, and evidence path

### Next

- exact next eligible atomic step
- human input or approval required, if any

## Appendix A. Canonical Repository Tree

```

DESS-ELTS-Research-Software/
|-- README.md
|-- AGENTS.md
|-- CLAUDE.md
|-- PROJECT_STATE.json           # generated
|-- .editorconfig
|-- .gitattributes
|-- .gitignore
|-- .github/
|   |-- CODEOWNERS
|   |-- pull_request_template.md
|   |-- ISSUE_TEMPLATE/
|   `-- workflows/
|       |-- repository-policy.yml
|       |-- schemas-progress.yml
|       |-- python-tests.yml
|       |-- unity-tests.yml
|       `-- release.yml
|-- unity/

```

```
|-- Assets/
|   |-- ELTS/{Clock,Config,Tracking,Geometry,Logging,Rendering,Scenario,Session,Calibration,Elts,Operator}/
|   |-- Tests/{EditMode,PlayMode}/
|   |-- ThirdParty/OpenVR/{LICENSE,NOTICE,checksums.sha256,...}
|   `-- StreamingAssets/config-generated/ # generated only
|-- Packages/{manifest.json,packages-lock.json}
`-- ProjectSettings/
|-- jetson/
|   |-- src/elts_link/
|   |-- tests/
|   |-- config/
|   |-- systemd/
|   `-- scripts/{install.sh,doctor.sh}
|-- analysis/
|   |-- src/elts_analysis/
|   |-- tests/fixtures/synthetic/
|   |-- pyproject.toml
|   `-- uv.lock
|-- config/
|   |-- defaults/
|   |-- study/{scenarios,sessions}/
|   |-- rig/templates/
|   |-- local.example.json
|   `-- local/ # ignored/generated
|-- schemas/
|   |-- config/
|   |-- logs/
|   |-- protocol/
|   |-- diagnostics/
|   |-- build/
|   `-- progress/
|-- scripts/
|   |-- bootstrap-dev.ps1
|   |-- doctor.ps1
|   |-- test.ps1
|   |-- build.ps1
|   |-- stage-config.ps1
|   |-- package-study.ps1
|   `-- setup-study-machine.ps1
|-- tools/progress/
|   |-- import_plan.py
|   |-- event.py
|   |-- reduce.py
|   |-- validate.py
|   `-- report.py
|-- project-management/
|   |-- baseline/{specification.docx,workbook.xlsx,SHA256SUMS}
|   |-- task-catalog.json
|   |-- progress/{events,handoffs,evidence-index.json}
|   `-- plan-changes/
|-- docs/
|   |-- architecture/
|   |-- adr/
|   |-- modules/
|   |-- ai/{context-index.md,agent-protocol.md,handoff-template.md}
|   |-- operator/
```

```
| |-- validation/
| |-- safety/
| |-- data-governance/
|-- licenses/
```

## Appendix B. PROJECT\_STATE.json Field Contract

| Field                       | Type         | Meaning                                                     |
|-----------------------------|--------------|-------------------------------------------------------------|
| \$schema                    | string       | Relative path to progress-state schema                      |
| schemaVersion               | integer      | Progress-state schema version                               |
| project                     | string       | Stable project name                                         |
| planVersion                 | string       | Preserved workbook/build-plan baseline                      |
| generatedAtUtc              | date-time    | UTC time produced by reducer; not event ordering authority  |
| generatedFrom.*Sha256       | sha256       | Hashes of preserved workbook/spec and normalized catalog    |
| generatedFrom.eventCount    | integer      | Count of accepted events                                    |
| current.mode                | enum         | working, blocked, verification, complete                    |
| current.phased / gated      | ID           | Earliest phase whose gate has not passed and its gate       |
| current.primaryStepId       | ID or null   | Deterministic exact current atomic step                     |
| current.activeStepIds       | ID array     | All valid in-progress steps in current phase                |
| current.nextEligibleStepIds | ID array     | Other ready steps after dependency evaluation               |
| current.blockingIds         | ID array     | Exact task, decision, gate, or external blockers            |
| activeWork[]                | object array | Owner, task, atomic step, branch, claims, started time      |
| tasks / decisions / gates   | maps         | All derived statuses, revisions, blockers, and evidence     |
| integrity                   | object       | Reducer version, source hashes, errors/warnings, valid flag |

## Appendix C. Initial P0.3 Bootstrap Atomic Steps

| Step      | Required outcome                                                                                                         |
|-----------|--------------------------------------------------------------------------------------------------------------------------|
| P0.3.S001 | Inventory the repository, branches, remotes, files, history, instructions, and working tree; preserve all existing work. |
| P0.3.S002 | Read and hash the specification, workbook, and available source plan; record baseline metadata.                          |
| P0.3.S003 | Create the bootstrap branch and declare task ownership, claimed paths, checks, and stop conditions.                      |
| P0.3.S004 | Copy original deliverables unchanged to project-management/baseline/ and create SHA256SUMS.                              |
| P0.3.S005 | Create schemas for task catalog, events, state, evidence, handoff, and plan changes.                                     |
| P0.3.S006 | Implement workbook import and normalized CSV/JSON exports with stable-ID/count reconciliation.                           |
| P0.3.S007 | Implement the append-only event writer with atomic creation and validation.                                              |
| P0.3.S008 | Implement the deterministic reducer and exact-current-step algorithm.                                                    |
| P0.3.S009 | Implement transition, dependency, ownership, event-chain, gate-authority, and evidence validation.                       |
| P0.3.S010 | Add reducer fixtures for baseline, active, verification, blocked, conflict, reopen, and completed states.                |
| P0.3.S011 | Seed the truthful bootstrap event and generate PROJECT_STATE.json; verify a no-diff second reduction.                    |
| P0.3.S012 | Create the canonical monorepo directories and root portability/policy files.                                             |
| P0.3.S013 | Create README.md and link the human quick start, architecture, workbook/catalog, state, and safety authorities.          |
| P0.3.S014 | Create canonical AGENTS.md and thin CLAUDE/Copilot adapters without duplicated policy.                                   |

| Step      | Required outcome                                                                                                       |
|-----------|------------------------------------------------------------------------------------------------------------------------|
| P0.3.S015 | Create ADR, module-contract, agent-handoff, evidence, validation, and operator-document templates.                     |
| P0.3.S016 | Create centralized toolchain/dependency manifest and choose the exact Unity patch from verified reality.               |
| P0.3.S017 | Create config source/staging architecture, schemas, examples, local whitelist, ignore rules, and divergence checks.    |
| P0.3.S018 | Create portable idempotent bootstrap, doctor, test, build, staging, package, and first-run script skeletons.           |
| P0.3.S019 | Create the Unity 6 URP project, remove XR packages, and commit settings, manifests, locks, and meta files.             |
| P0.3.S020 | Vendor/pin OpenVR binding and native library with version, origin, license, notice, and checksums.                     |
| P0.3.S021 | Add Newtonsoft JSON and Unity Test Framework through pinned package manifests.                                         |
| P0.3.S022 | Create EditMode and PlayMode assemblies with one meaningful smoke test each.                                           |
| P0.3.S023 | Create explicit Windows standalone command-line build entry and build-info generation.                                 |
| P0.3.S024 | Create CI for repository policy, schemas/progress, documentation, Python baseline, and Unity tests/build as available. |
| P0.3.S025 | Configure or document required GitHub ruleset, CODEOWNERS, PR template, squash merge, and mirror reconciliation.       |
| P0.3.S026 | Perform clean-clone/path-with-spaces/repeated-bootstrap/tests/build verification; record evidence and handoff.         |

## Appendix D. Decision Register D-01 through D-18

| ID   | Decision                                            | Owner                                     | Latest point    |
|------|-----------------------------------------------------|-------------------------------------------|-----------------|
| D-01 | Display type, size, refresh rate                    | Investigators                             | Phase 7 entry   |
| D-02 | Room layout, mounting points, base-station position | Investigators + developer                 | Phase 7 entry   |
| D-03 | Unity-Jetson transport                              | Developer + ELTS owner                    | Phase 6 entry   |
| D-04 | Moving-target movement specification                | Investigators                             | Phase 8 / P8.1  |
| D-05 | Shot-model parameters                               | Investigators                             | Phase 8 / P8.1  |
| D-06 | Participant-facing feedback                         | Investigators                             | Phase 8 / P8.1  |
| D-07 | Practice-session content                            | Investigators                             | Phase 8 / P8.1  |
| D-08 | Block-order assignment method and N=55 sheet        | Investigators                             | Phase 8 / P8.1  |
| D-09 | Mock weapon platform and sight                      | Investigators + fabricator                | Phase 1 / P1.11 |
| D-10 | ELTS behavior during NE blocks                      | Investigators                             | Phase 6 entry   |
| D-11 | Display luminance measurement method                | Investigators + developer                 | Phase 7 entry   |
| D-12 | Cross-device time synchronization                   | Collaborators + developer                 | Phase 8 / P8.4  |
| D-13 | Photobiological safety classification/basis         | Investigators                             | Phase 7 / P7.8  |
| D-14 | LED wattage, CCT, beam, and optics documentation    | ELTS builder                              | Phase 7 / P7.8  |
| D-15 | Center camera turret pans or remains fixed          | ELTS builder                              | Informational   |
| D-16 | Remaining scenario parameters                       | Investigators                             | Phase 8 / P8.1  |
| D-17 | Acceptance thresholds                               | Developer proposes; investigators approve | Phase 7 / P7.9  |
| D-18 | Tracking-hardware procurement confirmation          | Investigators                             | Phase 0         |

These identifiers are permanent. Accepted ADRs reference them; they do not replace or renumber them. Placeholder configuration values must be visibly invalid or labeled as non-decision defaults.

## Appendix E. Normative Requirement Catalog

This appendix contains 108 normative requirements. The companion workbook carries the same IDs plus implementation, verification, status, and evidence columns. CI must reconcile the two representations.

### E.1 Scientific validity

| ID      | Level    | Requirement                                                                                                                        | Implemented by             | Verified by |
|---------|----------|------------------------------------------------------------------------------------------------------------------------------------|----------------------------|-------------|
| SCI-001 | MUST     | Continuous aim error is the world-space angular error in degrees between the zero-corrected bore ray and the muzzle-to-target ray. | P2.4, P2.7, P4.5           | G2, G4, G8  |
| SCI-002 | MUST     | A target's canonical position is world-space; screen coordinates are derived each frame and never canonical target state.          | P2.7, P4.2                 | G4          |
| SCI-003 | MUST NOT | Unity sends head pose, target coordinates, turret angles, or individual LED commands to ELTS; only lifecycle commands are allowed. | P6.1, P6.5, P6.7           | G6          |
| SCI-004 | MUST     | The study uses no HMD and nothing near the eyes beyond approved measurement equipment.                                             | P1.2, P1.14                | G1, G8      |
| SCI-005 | MUST     | All application timestamps derive from one monotonic clock started at application launch; UTC is only an alignment anchor.         | P2.2, P6.3                 | G2, G6      |
| SCI-006 | MUST     | Every tracking sample contains connection and validity flags; invalid poses are excluded and never interpolated.                   | P2.3, P2.5, P2.8           | G2, G8      |
| SCI-007 | MUST     | Trackers bind by configured serial number, never device index.                                                                     | P1.3, P1.5, P2.3           | G1, G8      |
| SCI-008 | MUST     | Logged tracking uses zero prediction; rendering-only head prediction is isolated and the weapon pose remains raw.                  | P1.6, P2.3, P3.6           | G1, G3      |
| SCI-009 | MUST     | Trigger handling accepts the first falling edge and applies the configured lockout, nominally 20 ms.                               | P1.8, P2.3                 | G2          |
| SCI-010 | MUST     | The primary block DV remains the count of TargetDestroyed events during the fixed 300-second block.                                | P4.6, P4.8                 | G4, G8      |
| SCI-011 | MUST     | Scenario, behavior, scoring, session, geometry, display, and ELTS transport choices remain data-driven or interface-driven.        | P2.1, P4.1:P4.8, P6.1:P6.2 | G4, G6      |
| SCI-012 | MUST     | Participant and operator rendering remain independent; operator content never reaches the participant display.                     | P3.2, P3.4, P3.5           | G3          |

### E.2 Safety

| ID       | Level    | Requirement                                                                                                            | Implemented by    | Verified by |
|----------|----------|------------------------------------------------------------------------------------------------------------------------|-------------------|-------------|
| SAFE-001 | MUST     | Heartbeat timeout, maximum on-time, and normally closed physical E-stop operate as independent safeguards.             | P6.8, P6.12       | G6, G7      |
| SAFE-002 | MUST NOT | Software overrides or weakens the physical E-stop.                                                                     | P6.8, P6.12       | G6          |
| SAFE-003 | MUST     | Safety-critical ELTS changes receive human review and repeat the applicable fail-safe and bench tests.                 | P6.8, P7.8        | G6, G7      |
| SAFE-004 | MUST     | The LED exposure basis and operational limits are documented and agree with software guards.                           | P0.6, P7.8        | G7          |
| SAFE-005 | MUST     | The mock weapon is inert, visibly marked, secured, and covered by campus notification procedures.                      | P0.6, P1.11:P1.13 | G0, G8      |
| SAFE-006 | MUST     | A failed daily registration check stops participant operation until recalibration and incident recording are complete. | P5.9, P8.2        | G5, G8      |

### E.3 Portability

| ID      | Level    | Requirement                                                            | Implemented by | Verified by          |
|---------|----------|------------------------------------------------------------------------|----------------|----------------------|
| POR-001 | MUST NOT | Committed code or configuration contains user-specific absolute paths. | P0.3           | repository-policy CI |

| ID      | Level      | Requirement                                                                                                               | Implemented by | Verified by                 |
|---------|------------|---------------------------------------------------------------------------------------------------------------------------|----------------|-----------------------------|
| POR-002 | MUST       | Release scripts and runtime work from arbitrary writable extraction paths, including paths with spaces.                   | P0.3, P8.7     | clean-path portability test |
| POR-003 | MUST       | Bootstrap and first-run operations are idempotent.                                                                        | P0.3           | repeat-run test             |
| POR-004 | MUST       | Provisioning backs up existing local configuration or calibration before replacement.                                     | P0.3, P5.4     | provisioning test           |
| POR-005 | MUST NOT   | The study runtime requires the Unity Editor.                                                                              | P0.3, P8.7     | G8                          |
| POR-006 | SHOULD NOT | Prepared study operation requires internet connectivity.                                                                  | P0.3, P8.7     | offline rehearsal           |
| POR-007 | MUST       | Diagnostics emit both readable output and a versioned machine-readable report.                                            | P0.3, P4.12    | doctor schema test          |
| POR-008 | MUST       | A new physical rig receives a new rig identity and required recalibration instead of inheriting old calibration silently. | P5.4, P7.1     | G5, G7                      |
| POR-009 | MUST       | Unbundled prerequisites are precisely documented and detected.                                                            | P0.2, P0.3     | doctor checks               |
| POR-010 | MUST       | Every study release is verified after copy and extraction, not only in the build directory.                               | P8.7           | release smoke test          |

## E.4 Configuration

| ID      | Level    | Requirement                                                                                                    | Implemented by   | Verified by          |
|---------|----------|----------------------------------------------------------------------------------------------------------------|------------------|----------------------|
| CFG-001 | MUST     | Every configuration type has a versioned JSON Schema.                                                          | P0.3, P2.1       | schema CI            |
| CFG-002 | MUST     | Unknown, missing, or mistyped required fields cause a precise startup failure.                                 | P2.1             | G2                   |
| CFG-003 | MUST     | Every effective configuration file is hashed and recorded in logs and build provenance.                        | P2.1, P2.5, P8.7 | G2, G8               |
| CFG-004 | MUST     | local.json is generated from a committed example and contains only whitelisted machine-local fields.           | P0.3, P3.5       | config validation    |
| CFG-005 | MUST NOT | local configuration, participant calibration, participant data, credentials, or machine secrets are committed. | P0.3, P8.3       | repository-policy CI |
| CFG-006 | MUST NOT | Study settings and machine-local settings are mixed in one layer.                                              | P0.3, P2.1       | schema review        |
| CFG-007 | MUST     | A schema change increments a version and provides a migration or declares incompatibility.                     | P2.1, P2.8       | compatibility tests  |
| CFG-008 | MUST     | Configuration precedence is deterministic and documented.                                                      | P0.3, P2.1       | configuration tests  |
| CFG-009 | MUST     | The operator can inspect effective configuration and hashes before a session.                                  | P4.11, P4.12     | G4                   |
| CFG-010 | MUST     | A frozen study configuration change during data collection creates a new tag and study-log entry.              | P8.7             | release policy       |

## E.5 Repository structure

| ID      | Level      | Requirement                                                                                               | Implemented by | Verified by             |
|---------|------------|-----------------------------------------------------------------------------------------------------------|----------------|-------------------------|
| REP-001 | MUST       | The system uses one monorepo for Unity, Jetson, analysis, config, tooling, documentation, and governance. | P0.3           | repository layout check |
| REP-002 | MUST       | Root config is the sole hand-edited source; Unity StreamingAssets receives a generated staged copy.       | P0.3, P2.1     | config divergence CI    |
| REP-003 | SHOULD NOT | Git submodules are used before study-v1.0.                                                                | P0.3           | repository policy       |
| REP-004 | MUST       | Vendored OpenVR files include upstream version, license, and checksums.                                   | P0.3           | third-party audit       |
| REP-005 | MUST       | Unity ProjectSettings, package manifests, lockfiles, and all .meta files are committed.                   | P0.3           | Unity policy check      |
| REP-006 | MUST NOT   | Unity Library, Temp, Logs, obj, Build, Builds, or UserSettings directories are committed.                 | P0.3           | repository-policy CI    |

| ID      | Level | Requirement                                                                                        | Implemented by | Verified by                |
|---------|-------|----------------------------------------------------------------------------------------------------|----------------|----------------------------|
| REP-007 | MUST  | Binary baseline artifacts also receive diffable normalized exports for agents and review.          | P0.3           | ID and hash reconciliation |
| REP-008 | MUST  | Repository paths and scripts use portable relative resolution from the repository or release root. | P0.3           | path portability test      |

## E.6 Build and reproducibility

| ID        | Level | Requirement                                                                                                         | Implemented by | Verified by           |
|-----------|-------|---------------------------------------------------------------------------------------------------------------------|----------------|-----------------------|
| BUILD-001 | MUST  | The exact Unity 6 LTS editor version is pinned and checked.                                                         | P0.3           | doctor and build log  |
| BUILD-002 | MUST  | Unity and Python dependencies are locked with committed lockfiles.                                                  | P0.3, P2.8     | lockfile CI           |
| BUILD-003 | MUST  | Study releases are built from a clean Git worktree.                                                                 | P8.7           | release workflow      |
| BUILD-004 | MUST  | Dirty development builds identify themselves and cannot be used for participant data collection.                    | P0.3, P8.7     | build metadata check  |
| BUILD-005 | MUST  | The build records tag, commit, versions, schemas, configuration hashes, target, UTC, and artifact checksum.         | P0.3, P8.7     | build-info validation |
| BUILD-006 | MUST  | The release contains FIRST-RUN, CHECK-SYSTEM, START-ELTS, operator docs, licenses, build-info, and MANIFEST.sha256. | P8.7           | release layout test   |
| BUILD-007 | MUST  | CI specifies an explicit Unity build target or profile for reliable command-line builds.                            | P0.3           | Unity CI              |
| BUILD-008 | MUST  | Reproducibility claims are functional and provenance-based unless bit-for-bit determinism is separately proven.     | P0.3, P8.7     | release documentation |

## E.7 GitHub workflow

| ID      | Level    | Requirement                                                                              | Implemented by   | Verified by         |
|---------|----------|------------------------------------------------------------------------------------------|------------------|---------------------|
| GIT-001 | MUST     | main represents coherent, tested, documented work.                                       | P0.3             | protected ruleset   |
| GIT-002 | MUST     | Normal changes reach main through a pull request and required checks.                    | P0.3             | ruleset             |
| GIT-003 | MUST     | One task maps to one temporary branch and one active owner.                              | P0.3             | progress validator  |
| GIT-004 | SHOULD   | Pull requests use squash merge and merged branches are deleted.                          | P0.3             | repository settings |
| GIT-005 | MUST NOT | The same task branch is edited simultaneously on multiple machines.                      | P0.3             | contribution policy |
| GIT-006 | MUST     | A conflict is resolved deliberately and all affected checks are rerun.                   | P0.3             | review checklist    |
| GIT-007 | MUST NOT | Force pushes or deletion of main are allowed.                                            | P0.3             | ruleset             |
| GIT-008 | MUST     | Safety, invariants, frozen configuration, and study release tags receive human approval. | P6.8, P7.8, P8.7 | protected review    |

## E.8 AI-agent governance

| ID     | Level    | Requirement                                                                        | Implemented by | Verified by               |
|--------|----------|------------------------------------------------------------------------------------|----------------|---------------------------|
| AI-001 | MUST     | An agent reads the canonical instruction chain before editing.                     | P0.3           | handoff and PR checklist  |
| AI-002 | MUST     | An agent inspects PROJECT_STATE.json and its assigned atomic-step row before work. | P0.3           | progress event validation |
| AI-003 | MUST     | An agent declares task, branch, owner, scope, claimed paths, and stop conditions.  | P0.3           | task_started event        |
| AI-004 | MUST NOT | An agent changes unrelated files merely to clean up the repository.                | P0.3           | review                    |
| AI-005 | MUST NOT | An agent invents values for unresolved D-01 through D-18 decisions.                | P0.3:P8.1      | decision validation       |

| ID     | Level    | Requirement                                                                                                               | Implemented by           | Verified by                |
|--------|----------|---------------------------------------------------------------------------------------------------------------------------|--------------------------|----------------------------|
| AI-006 | MUST     | An invariant or authority conflict becomes a recorded blocker for human resolution.                                       | P0.3                     | blocked event              |
| AI-007 | MUST     | Agents preserve user changes and avoid destructive Git operations.                                                        | P0.3                     | review                     |
| AI-008 | MUST     | Interface, schema, configuration, or observable behavior changes update tests and documentation in the same pull request. | all implementation tasks | PR checklist               |
| AI-009 | MUST NOT | Work is marked done without required evidence.                                                                            | all tasks                | state reducer              |
| AI-010 | MUST     | Agents record commands and checks run, results, and evidence paths.                                                       | all tasks                | verification event         |
| AI-011 | MUST NOT | An agent merges or releases its own work without required human authority.                                                | P0.3, P8.7               | ruleset                    |
| AI-012 | MUST NOT | Two agents own the same task or branch concurrently.                                                                      | P0.3                     | event-chain conflict check |
| AI-013 | SHOULD   | Parallel agents use separate worktrees and non-overlapping module scopes.                                                 | P0.3                     | coordination policy        |
| AI-014 | MUST     | A handoff records completed work, remaining work, changed files, tests, risks, blockers, and next eligible step.          | P0.3                     | handoff schema             |
| AI-015 | MUST     | Trello and GitHub Issues are coordination mirrors; repository plan, events, evidence, and decisions remain canonical.     | P0.3, P0.4               | reconciliation check       |

## E.9 Progress state

| ID        | Level    | Requirement                                                                                                               | Implemented by      | Verified by        |
|-----------|----------|---------------------------------------------------------------------------------------------------------------------------|---------------------|--------------------|
| STATE-001 | MUST     | The immutable workbook is normalized into a versioned task catalog with stable IDs.                                       | P0.3                | catalog schema     |
| STATE-002 | MUST     | Progress changes are append-only events with unique IDs and actor, time, branch, transition, and evidence fields.         | P0.3                | event schema       |
| STATE-003 | MUST     | PROJECT_STATE.json is generated deterministically from the task catalog and events.                                       | P0.3                | regeneration CI    |
| STATE-004 | MUST NOT | Agents hand-edit generated PROJECT_STATE.json after the reducer exists.                                                   | P0.3                | regeneration CI    |
| STATE-005 | MUST     | Task transitions follow the documented state machine and invalid transitions fail.                                        | P0.3                | reducer tests      |
| STATE-006 | MUST     | G0 through G8 are hard gates and require explicit evidence-backed pass events.                                            | G0:G8               | gate reducer       |
| STATE-007 | MUST NOT | Completing tasks automatically passes a gate.                                                                             | G0:G8               | gate reducer       |
| STATE-008 | MUST     | The current step is chosen deterministically from active, verification-pending, ready, or blocked work in sequence order. | P0.3                | reducer tests      |
| STATE-009 | MUST     | Divergent event chains for one task fail reduction and require reconciliation.                                            | P0.3                | conflict test      |
| STATE-010 | MUST     | Done or passed states carry evidence that can be resolved in Git, CI, or approved lab records.                            | all tasks and gates | evidence validator |
| STATE-011 | MUST     | The generated state records source artifact hashes, plan version, event count, and generation time.                       | P0.3                | state schema       |
| STATE-012 | MUST     | If all eligible work is blocked, state mode is blocked and lists exact blocker IDs.                                       | P0.3                | reducer tests      |

## E.10 Documentation

| ID      | Level | Requirement                                                                                                                                                             | Implemented by              | Verified by               |
|---------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|---------------------------|
| DOC-001 | MUST  | AGENTS.md is the canonical model-agnostic instruction entry point.                                                                                                      | P0.3                        | documentation link check  |
| DOC-002 | MUST  | Model-specific instruction files are thin adapters and do not duplicate policy.                                                                                         | P0.3                        | documentation drift check |
| DOC-003 | MUST  | Each module contract states purpose, boundaries, interfaces, thread ownership, schemas, config, events, failure behavior, tests, invariants, ADRs, and work-item links. | module implementation tasks | module review             |

| ID      | Level    | Requirement                                                                                               | Implemented by    | Verified by       |
|---------|----------|-----------------------------------------------------------------------------------------------------------|-------------------|-------------------|
| DOC-004 | MUST     | Architecture decisions use immutable ADR history with Proposed, Accepted, Rejected, or Superseded status. | P0.3, decisions   | ADR lint          |
| DOC-005 | MUST NOT | D-01 through D-18 are renumbered or replaced by ADR identifiers.                                          | all decision work | ID reconciliation |
| DOC-006 | MUST     | Documentation changes occur in the same pull request as the behavior they describe.                       | all tasks         | PR checklist      |

## E.11 Research data

| ID       | Level    | Requirement                                                                                          | Implemented by | Verified by            |
|----------|----------|------------------------------------------------------------------------------------------------------|----------------|------------------------|
| DATA-001 | MUST NOT | Participant names, contact details, consent records, calibration, or study data enter Git or GitHub. | P0.3, P8.3     | data leak scan         |
| DATA-002 | MUST     | De-identified participant IDs are treated as research data and kept out of the public repository.    | P8.3           | data governance review |
| DATA-003 | MUST     | Raw study data is append-only; reruns use a unique suffix and preserve prior output.                 | P2.5, P4.10    | G2, G4                 |
| DATA-004 | MUST     | Only synthetic fixtures approved for public disclosure may be committed.                             | P2.9, P2.8     | repository-policy CI   |
| DATA-005 | MUST     | Logs record application, commit, schema, config, scenario, session, and timing provenance.           | P2.5           | G2                     |
| DATA-006 | MUST     | Backups include per-session checksums and follow IRB and institutional retention rules.              | P8.3           | G8                     |
| DATA-007 | MUST NOT | Study operation sends telemetry or research data to unapproved cloud services.                       | P0.3, P8.3     | security review        |
| DATA-008 | MUST     | Analysis preserves raw data, performs version-aware ingest, and records migrations.                  | P2.8, P8.6     | analysis tests         |

## E.12 Release management

| ID      | Level | Requirement                                                                                                   | Implemented by | Verified by                  |
|---------|-------|---------------------------------------------------------------------------------------------------------------|----------------|------------------------------|
| REL-001 | MUST  | The first data-collection release is tagged study-v1.0 after G8 passes.                                       | P8.7           | G8                           |
| REL-002 | MUST  | Study config, rig calibration, scenarios, and sessions are frozen by recorded hashes; local.json is excluded. | P8.7           | frozen manifest              |
| REL-003 | MUST  | A data-collection change creates a new tag and a study-log entry.                                             | P8.7           | release policy               |
| REL-004 | MUST  | The release archive includes checksum manifest, build-info, licenses, operator docs, and diagnostics.         | P8.7           | release verification         |
| REL-005 | MUST  | Release creation requires human approval and cannot substitute CI for hardware gate evidence.                 | P8.7           | release environment approval |

## Appendix F. Source and Evidence Register

| ID      | Source                                 | Location/version                                                                                                                                                                                                                                                                                  | Integrity/note                                                   |
|---------|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| SRC-001 | ELTS Build & Integration Plan          | Version 1.0, 4 September 2026                                                                                                                                                                                                                                                                     | 2EDA196256668AC23D5F80497ACB1A040BA8E28E1AA53FB6195C6AF75B71F01F |
| SRC-002 | DESS-ELTS-Research-Software repository | <a href="https://github.com/AceeEcho/DESS-ELTS-Research-Software">https://github.com/AceeEcho/DESS-ELTS-Research-Software</a>                                                                                                                                                                     | Live state must be rechecked at P0.3.S001                        |
| SRC-003 | GitHub - Available rules for rulesets  | <a href="https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets">https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets</a> | Official GitHub documentation; retrieved 4 September 2026        |

| ID      | Source                                         | Location/version                                                                                                                                                        | Integrity/note                                           |
|---------|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| SRC-004 | Unity 6 - Build a player from the command line | <a href="https://docs.unity3d.com/6000.0/Documentation/Manual/build-command-line.html">https://docs.unity3d.com/6000.0/Documentation/Manual/build-command-line.html</a> | Official Unity documentation; retrieved 4 September 2026 |
| SRC-005 | Companion implementation workbook              | ELTS_Implementation_and_Progress_Workbook.xlsx                                                                                                                          | Preserve and hash during repository bootstrap            |

## F.1 Assumptions requiring verification

- The repository may no longer be empty; inspect it live and preserve all current work.
- The exact Unity 6 LTS patch, Windows/GPU baseline, Jetson OS, OpenVR version, and transport are intentionally not invented here.
- GitHub ruleset features and enforcement can vary by repository/account plan; configure and test the effective policy.
- Private deployment, IRB, safety, and retention requirements may impose stricter controls than this public-repository baseline.
- All provisional hardware/performance thresholds from the source plan are finalized only through D-17 and Phase 7 evidence.

## End of Specification

Implementation may begin from P0.3.S001 using the companion workbook. No content in this specification authorizes participant data collection; that authorization exists only after the approved research process and an evidence-backed G8 pass for a tagged study release.

| Metric                      | Baseline value |
|-----------------------------|----------------|
| P-tasks                     | 91             |
| Atomic steps                | 243            |
| Decisions                   | 18             |
| Gate criteria               | 62             |
| Requirements                | 108            |
| Risks                       | 26             |
| Normalized dependency edges | 360            |
| First exact step            | P0.3.S001      |